Python 练习册 · 基础篇

打印提示：A4、实际大小（100%）、**关闭浏览器"页眉和页脚"**——本套教材的 PDF 版已自带页码，重复勾选会产生双页码；需要保留彩色底色时，在打印选项里开启"背景图形"。

本册关卡地图（先读这一页，1 分钟）

项目	内容
标签	**课程 + 字典** —— 课程（判定题）
本册关卡	挂在 3-00 ~ 3-10
用时	★30 题（约 10 小时，已并入阶段 3 关卡） + 3-00 前置关 3.3 小时
学完解锁	把 Python 语法练到能写脚本，同时当作阶段 3 各关的判定题

参考答案约占 15,600 字。做完之前不许翻；卡住 20 分钟以上再对答案。☆19 题留到以后再做。
另有前置关 **3-00（Python 基础快过验证）**：用本册阶段 1~4 的题快速验一遍，一眼就会的跳过。
关卡编号、时长、完成判定都写在正文每章/每节的「本关三要素」里；全书进度在《17-关卡总表》。

开始之前 · 先弄懂这是什么（1 分钟）

① **是什么**：这是一本 Python **练习册**（不是教科书）—— 把《04 Python 入门》的语法做成 **30 道 ★ 必做** + 19 道 ☆ 选做，每题都要你亲手把代码敲出来。类比：像学琴的**音阶练习**——谱子看懂了不等于手会，必须自己弹。

② **干什么用**："看懂"和"会写"是两回事。这一册把语法练到**不查笔记也能写出来**；它同时当 04 册阶段 3 各关的**判定题**（做出对应 ★ 题，那一关就算通过）。

③ **为什么要学（+ 实战里怎么用）**：实际工作里给的多是"写个脚本统计日志"这种小任务，考的就是当场能不能写出来；AI 运维要写的脚本（拉指标、清磁盘、调 API）正是这些小题拼起来的。

④ **长什么样 / 怎么用**：每题配四步式答案（先想什么 → 为什么这么写 → 完整代码 → 怎么验证）；规矩：**先自己想**（至少 3 分钟）、**真敲真跑**、卡住 20 分钟再翻答案。位置：04 册的配套练习，做完再进 06《进阶篇》。

目录

前言	定位、分级规则、自检规则、答题要求、安全红线
示范题	2 道（锁定"四步式答案"质量标准）
阶段1	变量与类型（8 题：5 必做 + 3 选做）
阶段2	控制流（8 题：5 必做 + 3 选做）
阶段3	数据结构（9 题：5 必做 + 4 选做）
阶段4	函数与模块（8 题：5 必做 + 3 选做）
阶段5	文件与异常（8 题：5 必做 + 3 选做） ☑ 检查点②
阶段6	模块组合（8 题：5 必做 + 3 选做）
参考答案	书末集中，每题四步式
附录	本册学习路线的对应关系

前言

1. 本书定位

学完《Python 入门》教科书后的**独立巩固体系**。顺序不可跳：教科书 → 本基础篇 → 进阶篇。

2. 题目分级（★必做 / ☆选做）

49 题全部标注等级，把"必须全做"的压力拆掉：

等级	范围	数量
★ **必做**	每个概念的变式 1 + 各阶段收尾综合题	每阶段约 5 题，共 30 题
☆ 选做	变式 2 中较难的一半 + 加深题	每阶段约 3 题，共 19 题

3. 自检规则（决定 ☆ 做不做）

做完一道 ★ 变式 1 后自检：**一次做对、用时少、全程独立没看答案**——则该概念的 ☆ 变式 2 可以跳过；**卡壳、做错、超时**——则该概念的 ☆ 必做，把漏洞补上。

做完之后：本册所有阶段做完，请打开《**17-关卡总表**》对照本册部分逐条打勾——**打勾 = 真正会了**（不是"看过"）；卡住的条目回到对应阶段重做。 ☑ 打勾全部完成，就可以进入下一册。

4. 用时预估

基础篇全程 **10~14 小时**：全做 49 题约 14h；按自检规则跳题约 10h。每阶段约 1.5~2 小时（必做约 1h + 选做约 0.5h）。

5. 答题要求（每道题都要遵守）

① 先**在纸上/脑子里想好思路**，再敲代码；② 每道题必须**自己跑通看到输出**；③ 对答案时**先看"先想什么"和"为什么这么写"**，最后才看代码；④ 做完的代码和心得记进当天学习日志。

6. 题型图例

题型	含义	要求
动手	从零写代码跑通	独立完成
观察	改代码看变化	解释现象
组合	串联多个知识点	综合运用
辨析	讲区别、讲为什么	口头能说清

🌐 = 须电脑环境（Ubuntu）完成；平板 Termux 环境下不确定。Windows 会报错。

实验闭环：先记录练习文件原状态；只在 `~/lab` 操作；运行后核对内容、行数和输出；结束后删除临时文件或恢复现场。异常时保留 traceback。

7. 练习环境与安全红线

电脑 Ubuntu（Python 3 自带）或平板 Termux 均可。**红线**：实验文件一律放在 `~/lab`；涉及**覆盖（w 模式）**、**删除**、**联网**的题，题面已加安全提示，请照提示执行；不访问陌生网站，联网题只用题面指定的公开接口。

示范题（先看这 2 道，锁定答案标准）

这两道示范题的"答案四步式"就是全书 49 题答案的标准。做题前先读一遍，了解"对答案时应该看到什么"。
（说明：两道示范题均为独立场景，不与任何正题重叠——它们的用途是展示"四步式答案"的写法标准）

示范题 1（独立示范，不占任何正题）

题目：让用户输入 3 行内容，逐行**追加**写入 `notes.txt`（先 `cd ~/lab` 再运行），最后打印"已写入 3 行"。

① 先想什么：三步拼装——`input` 读一行（第 2 章）、`open("a")` 追加写（第 6 章）、`for` 循环 3 次（第 2 章）。写进文件的每一行都要自己补换行符。

② 为什么这么写：`"a"` 模式 = 追加（只往末尾加，不覆盖原内容——第 6 章对比过 w 和 a）；`write` 不会自动加换行，所以要写 `"\n"`；`range(3)` 控制次数。

③ 完整代码：

```
with open("notes.txt", "a", encoding="utf-8") as f:
    for i in range(3):
        line = input("第 " + str(i+1) + " 行：")
        f.write(line + "\n")
print("已写入 3 行")
```

④ 怎么验证做对了：跑完 `cat notes.txt` 能看到你输入的 3 行；**再跑一轮**（再输 3 行），文件变成 6 行——证明 a 是"追加"不是"覆盖"。

示范题 2（独立示范，不占任何正题）

题目：写一个命令行小工具：`--times N`（次数）参数，把"你好"打印 N 次，并用 `logging` 记一条 INFO 日志。

① 先想什么：两个知识点组合——`argparse` 收参数（第 9 章）、`logging` 记日志（第 10 章）。流程：解析参数 → 循环打印 → 记一条日志。

② 为什么这么写：`type=int` 把参数自动转成数字（第 9 章）；`logging.basicConfig` 配好文件名、级别和格式（第 10 章）；`range(args.times)` 按次数循环。

③ 完整代码：

```
import argparse
import logging

logging.basicConfig(filename="hello.log", level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")

parser = argparse.ArgumentParser()
parser.add_argument("--times", type=int, required=True, help="打印次数")
args = parser.parse_args()

for i in range(args.times):
    print("你好")
logging.info("打印了 %s 次", args.times)
```

④ 怎么验证做对了：`python3 hello.py --times 3` 打印 3 次"你好"；`cat hello.log` 有一条 INFO；不传 `--times` 会报 required 错误；传 `--times abc` 会报类型错误——四种行为都对，就是真的懂了。

阶段 1｜变量与类型（约 1.5h）

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题——**当关做完，不许攒着。**
对应关卡：3-00（快过验证：★ 题在 20 道基础题范围内；并快过 04 的第 0~4 章）

判定写法：□ 能独立做出本阶段 ★ 题（题号见上表）。做完即算该关通过；☆ 题留到以后再做。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：熟练定义变量、操控字符串和数字、会格式化输出。

阶段产出：一个"自我介绍卡"程序（姓名/年龄/城市/进度条，用 f-string 输出）。

编号	题型	考点（教科书章）	题目（只给目标）	前置
1-1 ★	**动手**	print 多参数（第1章）	用一条 `print()`（打印）同时输出你的名字、年龄和所在城市，中间用 - 分隔	第1章
1-2 ☆	**动手**	print 参数（第1章）	打印"进度 50%"，要求 `print` 只用一个参数完成（提示：一个字符串里自带空格）	第1章
1-3 ★	**动手**	变量与重赋值（第1章）	定义变量 `score`（分数）为 60，输出一次；把它改成 90 再输出一次；打印两个结果的类型	第1章
1-4 ☆	**观察**	type（第1章）	观察 `type(5)`、`type(5.0)`、`type("5")` 的差别，用一句话总结	第1章
1-5 ★	**动手**	字符串操作（第1章）	用拼接、重复、`len()`（长度）生成一张"分割线卡片"（如 `====欢迎====`）	第1章
1-6 ☆	**动手**	f-string（第1章）	用 f-string 重做 1-1 题，并额外输出一位小数（如 3.1 而非 3.14159）	第1章
1-7 ★	**动手**	数字运算（第1章）	计算 `7/2`、`7//2`、`7%2`、`2**10` 并逐行打印，各配一句注释说明结果含义	第1章
1-8 ★	**辨析**	变量命名（第1章）	解释：为什么 `1name` 和 `if` 不能作变量名？分别违反了什么规则？	第1章

阶段 2｜控制流（约 1.5h）

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题——**当关做完，不许攒着。**
对应关卡：3-00（快过验证：★ 题在 20 道基础题范围内）

判定写法：□ 能独立做出本阶段 ★ 题（题号见上表）。做完即算该关通过；☆ 题留到以后再做。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：能用 if 分支和两种循环写出带判断的程序。

阶段产出：一个完整的"猜数字"游戏（含次数统计与再来一局）。

编号	题型	考点（教科书章）	题目（只给目标）	前置
2-1 ★	**动手**	if/else（第 2 章）	写程序：输入一个整数，输出它是正数、负数还是零	第2章
2-2 ☆	**动手**	if/elif（第 2 章）	成绩分段：≥90 优秀、≥75 良好、≥60 及格、其余不及格	第2章
2-3 ★	**动手**	for + range（第 2 章）	打印 1 到 100 中所有 7 的倍数	第2章
2-4 ☆	**动手**	for + enumerate（第 2 章 +《Python 内置函数手册》第 4 组）	给一份名单加编号输出（如 `1号：张三`），要求序号从 1 开始（提示：`enumerate()`（编号遍历）有第二个参数）	第2章
2-5 ★	**动手**	while 计数器（第 2 章）	用 `while` 打印 10 到 1 的倒计时，最后输出"发射！"	第2章
2-6 ★	**组合**	while + break（第 2 章）	猜数字：程序随机想一个 1~100 的数（提示：`random.randint`（随机整数；完整用法见《Python 入门》5.4 random 模块）），猜对为止，输出猜了几次	《Python 入门》第 2、5 章
2-7 ☆	**辨析**	for vs while（第 2 章）	解释：知道循环次数时用哪个？不知道次数（如猜数字）时用哪个？各举一例	第2章
2-8 ★	**组合**	嵌套循环（第 2 章）	打印九九乘法表，格式对齐	第2章

阶段 3｜数据结构（约 2h）

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题——**当关做完，不许攒着。**
对应关卡：3-00（快过验证：★ 题在 20 道基础题范围内）

判定写法：□ 能独立做出本阶段 ★ 题（题号见上表）。做完即算该关通过；☆ 题留到以后再做。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：用 list 与 dict（含 tuple/set）管理一组数据，会增删改查。

阶段产出：一个通讯录小程序（增删查全功能）。

编号	题型	考点（教科书章）	题目（只给目标）	前置
3-1 ★	**动手**	list 基本操作（第3章）	建一个空列表，依次放入 3 个元素，用负数索引取出最后一个	第3章
3-2 ☆	**动手**	list 切片（第3章）	对列表 `[0,1,2,3,4,5]` 分别取：前 3 个、后 2 个、全部（用切片写）	第3章
3-3 ★	**动手**	dict 增删改查（第3章）	建一个"英文→中文"的小词典，完成一次查词、加词、改词、删词	第3章
3-4 ☆	**动手**	dict get 与遍历（第3章）	用 `get()`（安全取值）查一个不存在的词并给默认提示"查无此词"；再遍历输出整个词典	第3章
3-5 ★	**动手**	tuple/set（第3章）	用 `set`（集合）给一份有重复的名单去重；解释为什么结果顺序可能变化	第3章
3-6 ★	**组合**	list + tuple（第3章）	存 3 名学生的"姓名+分数"，找出最高分并打印其姓名	第3章
3-7 ☆	**辨析**	list vs tuple（第3章）	解释：什么时候用 `tuple`（元组）而不是 list？给一个真实场景	第3章
3-8 ★	**组合**	通讯录（第3章）	写一个循环运行的通讯录：输入 1 添加、2 查询、3 退出	第3章
3-9 ☆	**组合**	字符串方法串烧（第1、3章+手册）	一句话完成：把输入的一行日志 `" 2026-08-25 14:30:12 ERROR 磁盘满 "` 去首尾空白、按空格切开、取第 3 块（日志级别）并小写化、判断它是否以 "err" 开头——strip / split / lower / startswith 四个字符串方法各用一次	《Python内置函数手册》· 字符串方法速查 + 04 册第1、3章

阶段 4｜函数与模块（约 1.5h）

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题——**当关做完，不许攒着。**
对应关卡：3-00（快过验证：★ 题在 20 道基础题范围内）

判定写法：□ 能独立做出本阶段 ★ 题（题号见上表）。做完即算该关通过；☆ 题留到以后再做。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：会写函数、传参、拿返回值，会导入模块。

阶段产出：一组自己写的 3 个函数（工具函数库雏形）。

编号	题型	考点（教科书章）	题目（只给目标）	前置
4-1 ★	**动手**	def 定义与调用（第4章）	写一个函数打印"我是新函数"，连续调用 3 次	第4章
4-2 ☆	**动手**	参数与默认参数（第4章）	写 `greet(name, greeting="你好")`，分别用默认值和自定义值调用	第4章
4-3 ★	**动手**	return 与多返回值（第4章）	写一个函数同时返回两个数中的较大值和较小值	第4章
4-4 ☆	**观察**	作用域（第4章）	在函数内给一个与外部同名的变量赋值，观察外部变量是否变化，解释原因	第4章
4-5 ★	**动手**	import os（第5章）	用 `os` 模块打印当前目录、列出目录下所有文件	第5章
4-6 ☆	**动手**	from/import as（第5章）	分别用 `from os import getcwd` 和 `import time as t` 两种写法各调用一个函数（打印当前目录 / 打印时间戳）	第5章
4-7 ★	**组合**	重构（第4章）	把阶段 3 通讯录的"添加"功能改造成一个函数，程序主体调用它	第4章
4-8 ★	**辨析**	print vs return（第4章）	写一个"有 return 无 print"的函数和"有 print 无 return"的函数，展示差异并总结	第4章

阶段 5｜文件与异常（约 2h）

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题——**当关做完，不许攒着。**
对应关卡：3-03 / 3-04 / 3-05
判定写法：□ 能独立做出本阶段 ★ 题（题号见上表）。做完即算该关通过；☆ 题留到以后再做。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：会读写文本和 JSON 文件，会给程序加异常处理。 ☑ 本阶段完成 = 达成检查点②。

阶段产出：一个能读写文件、带异常处理的通讯录工具。

📌 **本阶段统一实验约定**：① **初始状态**：文件题先建练习目录并用答案③给的初始化命令创建初始文件（5-1 造 test.txt；5-3 自建 data.json；5-4 自建 contacts.json）；② **是否依赖前题**：5-2 自建 log.txt；5-7 使用 5-1 的 test.txt（或自建），其余各题自建素材、互不依赖；③ **重复运行影响**：w 模式会清空重写、a 模式会追加——重复运行前按答案④清理测试文件，否则行数/内容叠加会误判；④ **清理恢复**：做完本阶段删除 ~/lab 里的测试文件（test.txt、data.json、contacts.json 等），恢复练习前状态。

编号	题型	考点（教科书章）	题目（只给目标）	前置
5-1 ★	**动手**	open 读文件（第6章）	把一个文本文件整段读出，再逐行读出，各打印一次（实验文件自建，放 ~/lab 下）	第6章
5-2 ☆	**动手**	写入模式（第6章）	分别用 "w" 和 "a" 模式向同一文件写两次内容，观察并解释最终文件内容（⚠ w 会清空原内容，只用练习文件做）	第6章
5-3 ★	**动手**	json 读写（第6章）	把一个 dict 存成 JSON 文件，再读回来并打印（⚠ 写入用 ~/lab 下的练习文件）	第6章
5-4 ☆	**动手**	json + 通讯录（第6章）	把阶段 3 通讯录改造成"退出时保存到 JSON、启动时读回"	第6章
5-5 ★	**动手**	try/except（第7章）	读一个不存在的文件，用 `try/except` 捕获 `FileNotFoundError` 并输出友好提示	第7章
5-6 ☆	**动手**	finally（第7章）	在 5-5 基础上加 `finally`，无论成功失败都输出"本次读取结束"	第7章
5-7 ★	**组合**	读文件统计（第6章）	统计一个文本文件的行数、总字数，输出一句话报告	第6章
5-8 ★	**辨析**	w vs a（第6章）	解释 "w" 覆盖与 "a" 追加的区别，并说出它们分别对应 Linux 的哪个符号（> / >>）	第6章

阶段 6｜模块组合（约 2h）

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题——**当关做完，不许攒着。**
对应关卡：3-08 / 3-09 / 3-10

判定写法：□ 能独立做出本阶段 ★ 题（题号见上表）。做完即算该关通过；☆ 题留到以后再做。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：把 requests/argparse/logging/subprocess 组合成命令行工具。
阶段产出：一个带参数、带日志的命令行小工具（本阶段旗舰题 6-8）。**外部条件**：6-1、6-2 需网络和 requests（断网/未安装时跳过，不影响主线，主线为 6-3~6-8）。

编号	题型	考点（教科书章）	题目（只给目标）	前置
6-1 ☆	**动手**	requests 基本（第8章）	请求一个公开 API，打印状态码和返回的 JSON（⚠ 外部扩展：需网络 + requests，断网/未安装就跳过，不影响主线）	第8章
6-2 ☆	**动手**	requests 异常（第8章）	请求一个连不上的地址（不存在的主机，或本机未监听的端口），用 `try/except` 接住并输出"网络不通"（提示：`requests.exceptions.RequestException` = requests 所有网络异常的基类，接住它 = 接住"连不上"）	第8章
6-3 ★	**动手**	argparse 参数（第9章）	写一个脚本，接收 `--name` 参数并打印"你好，{name}"	第9章
6-4 ★	**动手**	argparse 默认值（第9章）	在 6-3 基础上：不传参数时用默认名字，并让 `--help` 有中文说明	第9章
6-5 ★	**动手**	logging 基本（第10章）	给一个脚本配日志，INFO 级别记录"开始/结束"，ERROR 记录异常，输出到文件	第10章
6-6 ☆	**观察**	logging 级别（第10章）	把日志级别改成 WARNING，观察 INFO 日志消失，解释级别过滤机制	第10章
6-7 ★	**动手**	subprocess 基本（第11章）	用 Python 运行 `df -h`（查磁盘）和 `free -h`（查内存），捕获并打印输出（🌐 本命令为 Linux 命令，需在 Ubuntu 环境运行（Termux 下命令不全），Windows 会报错）	第11章
6-8 ★	**组合**	三模块综合（第 9/10/11 章：argparse+logging+subprocess）	写一个命令行小工具：`--check disk` 查磁盘、`--check mem` 查内存，操作记入日志（🌐 同 6-7：df/free 为 Linux 命令，需 Ubuntu 环境（Termux 下命令不全））	第 9/10/11 章

（本叠是题目，参考答案在另一叠；卡住 20 分钟以上再翻）